03-data-model

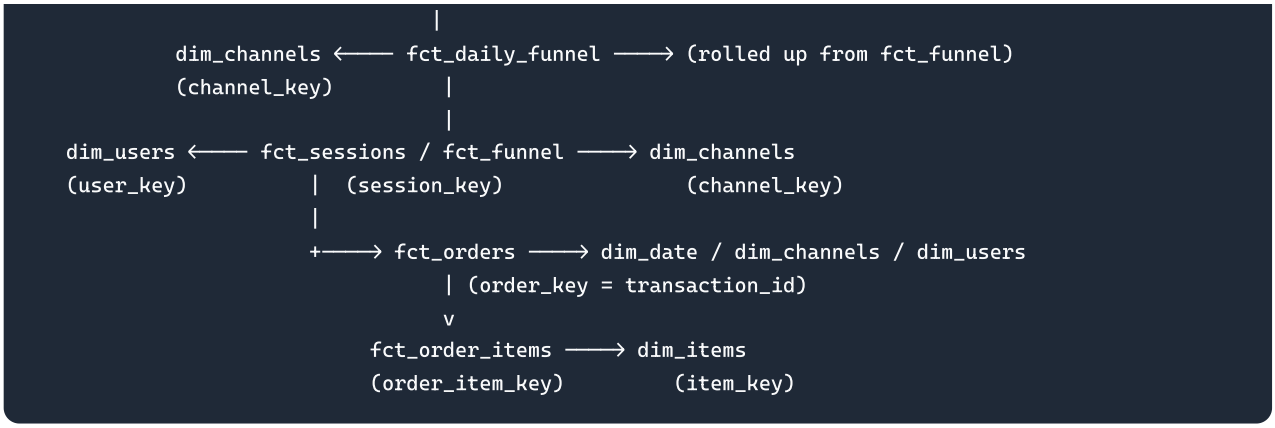
8. Data Model

Helios models the GA4 export as a four-layer dbt DAG: **raw GA4 -> staging -> intermediate -> marts**. Every transformation is governed; the LLM never touches raw events directly. It composes governed metrics via `semantic-mcp`, which itself only references the marts described here. This separation is what guarantees the FOUNDATION principle "grounding over generation" and the success target of **0 hallucinated columns / 100% governed SQL**.

8.1 Layer Overview

Staging models are materialized as `view` (cheap, always-fresh casts). Intermediate models are `ephemeral` or `view`. Marts are materialized as `table` (or incremental on `event_date` for the large fact tables) so that the autonomous run hits a fixed, predictable BigQuery byte budget.

8.2 Star Schema Diagram



`fct_funnel` is the grain-defining session fact; `fct_orders` is the grain-defining transaction fact. All dimensions conform (shared `dim_date`, `dim_channels`, `dim_users`) so the Decompose agent can pivot any metric across `device_category`, `channel_group`, `country`, etc.

8.3 Mart Catalog — Fact Tables

fct_sessions

- **Grain:** one row per session.
- **Primary key:** `session_key`.
- **Description:** session-level attributes, volumes, engagement.

column	type	key	description
session_key	STRING	PK	`to_hex(md5(user_pseudo_id
user_pseudo_id	STRING	FK -> dim_users	device/cookie key (de facto user)
ga_session_id	INT64		session id within user
date_key	DATE	FK -> dim_date	session start date (from min event_timestamp)
session_start_ts	TIMESTAMP		first event_timestamp of session
session_end_ts	TIMESTAMP		last event_timestamp of session
channel_key	STRING	FK -> dim_channels	session-scoped channel group surrogate
source	STRING		session-scoped source
medium	STRING		session-scoped medium
campaign	STRING		session-scoped campaign

column	type	key	description
landing_page	STRING		first `page_location` of session
device_category	STRING		mobile / desktop / tablet
operating_system	STRING		device.operating_system
browser	STRING		device.web_info.browser
country	STRING		geo.country
region	STRING		geo.region
is_new_user	BOOL		TRUE if ga_session_number = 1
ga_session_number	INT64		session ordinal for the user
event_count	INT64		events in session
engaged_session	BOOL		session_engaged = "1" OR engagement_time_msec >= 10000
engagement_time_msec	INT64		summed engagement time

fct_funnel

- **Grain:** one row per session.
- **Primary key:** `session_key`.
- **Description:** the canonical session fact carrying the macro-funnel boolean flags plus per-session revenue. Joins 1:1 to `fct_sessions`.

column	type	key	description
session_key	STRING	PK / FK -> fct_sessions	session id
user_pseudo_id	STRING	FK -> dim_users	user key
date_key	DATE	FK -> dim_date	session date
channel_key	STRING	FK -> dim_channels	channel group
device_category	STRING		dimension
country	STRING		dimension
is_new_user	BOOL		new vs returning

column	type	key	description
did_session_start	BOOL		always TRUE (denominator anchor)
reached_view_item	BOOL		session reached view_item or beyond (max-downstream)
reached_add_to_cart	BOOL		session reached add_to_cart or beyond (max-downstream)
reached_begin_checkout	BOOL		session reached begin_checkout or beyond (max-downstream)
reached_add_shipping_info	BOOL		session reached add_shipping_info or beyond (max-downstream)
reached_add_payment_info	BOOL		session reached add_payment_info or beyond (max-downstream)
reached_purchase	BOOL		session reached purchase (fired purchase)
session_revenue	FLOAT64		sum purchase_revenue_in_usd in session (deduped)

fct_daily_funnel

- **Grain:** one row per (`date_key` , `channel_key` , `device_category` , `country` , `is_new_user`).
- **Primary key:** composite of the grain columns (`daily_funnel_key` surrogate).
- **Description:** pre-aggregated daily funnel, the primary feed for the Monitor (time-series anomaly) and Decompose (mix-shift) agents. Aggregates `fct_funnel` .

column	type	description
daily_funnel_key	STRING	md5 of grain columns
date_key	DATE	day
channel_key	STRING	channel group
device_category	STRING	dimension
country	STRING	dimension
is_new_user	BOOL	dimension
sessions	INT64	count distinct session_key
users	INT64	count distinct user_pseudo_id

column	type	description
new_users	INT64	users where is_new_user
returning_users	INT64	users where not is_new_user
engaged_sessions	INT64	sessions where engaged_session
view_item_sessions	INT64	sum reached_view_item
add_to_cart_sessions	INT64	sum reached_add_to_cart
begin_checkout_sessions	INT64	sum reached_begin_checkout
add_shipping_info_sessions	INT64	sum reached_add_shipping_info
add_payment_info_sessions	INT64	sum reached_add_payment_info
purchasing_sessions	INT64	sum reached_purchase
transactions	INT64	distinct transaction_id
revenue	FLOAT64	sum session_revenue

Rates (`session_conversion_rate` , `view_to_cart_rate` , etc.) are NOT stored here; they are computed in the semantic layer from these additive counts so they stay re-aggregatable across any slice.

fct_orders

- **Grain:** one row per transaction (distinct `transaction_id`).
- **Primary key:** `order_key` = `transaction_id` .
- **Description:** deduped order header. One purchase event per transaction after dedup.

column	type	key	description
order_key	STRING	PK	transaction_id
session_key	STRING	FK -> fct_sessions	originating session
user_pseudo_id	STRING	FK -> dim_users	buyer
date_key	DATE	FK -> dim_date	purchase date
channel_key	STRING	FK -> dim_channels	attributed channel
order_ts	TIMESTAMP		purchase event timestamp
gross_revenue	FLOAT64		purchase_revenue_in_usd

column	type	key	description
refund_value_in_usd	FLOAT64		refund (usually 0/NULL)
net_revenue	FLOAT64		gross_revenue - coalesce(refund,0)
shipping_value_in_usd	FLOAT64		ecommerce.shipping_value_in_usd
tax_value_in_usd	FLOAT64		ecommerce.tax_value_in_usd
total_item_quantity	INT64		items in order
unique_items	INT64		distinct items in order

fct_order_items

- **Grain:** one row per (`transaction_id` , item line).
- **Primary key:** `order_item_key` = md5(transaction_id + item_id + row_number).
- **Description:** exploded `items[]` array for purchases. Feeds item-category diagnoses.

column	type	key	description
order_item_key	STRING	PK	surrogate
order_key	STRING	FK -> fct_orders	transaction
item_key	STRING	FK -> dim_items	item_id
item_name	STRING		items.item_name
item_category	STRING		items.item_category
quantity	INT64		items.quantity
item_revenue_in_usd	FLOAT64		items.item_revenue_in_usd
price_in_usd	FLOAT64		items.price_in_usd
coupon	STRING		items.coupon

8.4 Mart Catalog — Dimension Tables

table	grain / PK	key columns	description
dim_users	one row per user_pseudo_id (PK `user_key`)	first_touch_ts, first_touch_source, first_touch_medium, first_channel_key, total_sessions, total_revenue, is_purchaser	user roll-up; first-touch attribution from `traffic_source` struct

table	grain / PK	key columns	description
dim_items	one row per item_id (PK `item_key`)	item_name, item_brand, item_category, item_category2..5, current_price_in_usd	product catalog distilled from items[]
dim_channels	one row per channel_group (PK `channel_key`)	channel_group, channel_order	the 10 GA4 default channel groups
dim_date	one row per calendar day (PK `date_key`)	date_key, day, week, month, year, day_of_week, is_weekend	conformed date spine 2020-11-01..2021-01-31

8.5 Sessionization (in depth)

GA4 does not ship a session row; a session is reconstructed from events sharing the same `(user_pseudo_id, ga_session_id)`. `ga_session_id` lives inside `event_params` and must be unnested. The canonical surrogate is `session_key = md5(user_pseudo_id || '-' || ga_session_id)`.

```
-- int_ga4__sessionized
with params as (
  select
    user_pseudo_id,
    event_timestamp,
    event_name,
    (select ep.value.int_value    from unnest(event_params) ep where ep.key='ga_session_id')     as
        ga_session_id,
    (select ep.value.int_value    from unnest(event_params) ep where ep.key='ga_session_number') as
        ga_session_number,
    (select ep.value.string_value from unnest(event_params) ep where ep.key='page_location')      as
        page_location,
    (select coalesce(ep.value.string_value,'(direct)') from unnest(event_params) ep where
        ep.key='source') as ev_source,
    (select coalesce(ep.value.string_value,'(none)')   from unnest(event_params) ep where
        ep.key='medium') as ev_medium,
    (select ep.value.int_value    from unnest(event_params) ep where ep.key='engagement_time_msec')
        as engagement_time_msec,
    (select ep.value.string_value from unnest(event_params) ep where ep.key='session_engaged')
        as session_engaged,
    device.category as device_category, device.operating_system, device.web_info.browser,
    geo.country, geo.region, traffic_source.source as ut_source, traffic_source.medium as ut_medium
  from {{ source('src_ga4','events') }}
  where _table_suffix between '20201101' and '20210131'
)
select
  to_hex(md5(user_pseudo_id || '-' || cast(ga_session_id as string)))        as session_key,
  user_pseudo_id, ga_session_id, any_value(ga_session_number) as ga_session_number,
  min(event_timestamp) as session_start_micros,
  max(event_timestamp) as session_end_micros,
```

```
  -- landing_page = page_location of the earliest event carrying one
  array_agg(page_location ignore nulls order by event_timestamp limit 1)[safe_offset(0)] as
        landing_page,
  -- session-scoped source/medium: first non-null event-param value; fall back to user first-touch
  coalesce(array_agg(ev_source ignore nulls order by event_timestamp limit 1)[safe_offset(0)],
        any_value(ut_source), '(direct)') as source,
  coalesce(array_agg(ev_medium ignore nulls order by event_timestamp limit 1)[safe_offset(0)],
        any_value(ut_medium), '(none)')   as medium,
  any_value(device_category) as device_category, any_value(operating_system) as operating_system,
  any_value(browser) as browser, any_value(country) as country, any_value(region) as region,
  countif(true) as event_count,
  max(coalesce(engagement_time_msec,0)) as engagement_time_msec,
  logical_or(session_engaged='1') as session_engaged_flag
from params
where ga_session_id is not null
group by user_pseudo_id, ga_session_id
```

Engagement: `engaged_session = session_engaged_flag OR engagement_time_msec ≥ 10000`, matching GA4's engaged-session definition; `engagement_rate = engaged_sessions / sessions`. **Landing page** is the `page_location` of the earliest-timestamp event with a non-null value (typically the `session_start` / `first_visit` /first `page_view`).

The **traffic_source gotcha** is handled here explicitly: `traffic_source.*` is USER-LEVEL first-touch attribution, not session source. So Helios prefers the session-scoped `event_params.source/medium` (which reflect the actual acquisition of *this* session) and only falls back to the user-level `traffic_source` struct when the session params are null.

8.6 User Identity Resolution and Limits

The user key is `user_pseudo_id` (device/cookie id). `user_id` is almost always NULL in this obfuscated dataset, so cross-device stitching is impossible — a single human on phone + laptop appears as two users. `dim_users` therefore resolves identity at the device-cookie grain: first-touch timestamp = `min(user_first_touch_timestamp)`, first-touch channel from the `traffic_source` struct, and `is_new_user` derived from `ga_session_number = 1` / the `first_visit` event. Returning-user counts double-count cookie churn (cleared cookies look like new users), so all user-grain metrics (`new_users`, `returning_users`, `revenue_per_user`/ARPU) are caveated by the Critic agent as cookie-based approximations, never true person-level counts.

---

9. Event Model

9.1 Canonical Event Taxonomy

GA4 rows are events. The table below catalogs every canonical `event_name` present in the dataset, when it fires, its load-bearing params, and the funnel stage it maps to.

event_name	fires when	key event_params	funnel stage
session_start	first event of a session	ga_session_id, ga_session_number	session_start (anchor)
first_visit	first ever event for a user	ga_session_id	new-user flag
page_view	any page load	page_location, page_title, page_referrer	(engagement)
view_promotion	promo impression	items[], promotion_id	top of funnel
view_item_list	category/list page view	items[], item_list_name	top of funnel
view_item	product detail page view	items[], page_location	**view_item**
select_item	item clicked in a list	items[]	micro: list -> PDP
add_to_cart	item added to cart	items[], value, currency	**add_to_cart**
view_cart	cart viewed	items[], value	micro: cart
begin_checkout	checkout initiated	items[], value, coupon	**begin_checkout**
add_shipping_info	shipping tier entered	shipping_tier, value	**add_shipping_info**
add_payment_info	payment method entered	payment_type, value	**add_payment_info**
purchase	order completed	transaction_id, value, items[], ecommerce	**purchase**
scroll	90% scroll reached	percent_scrolled	engagement
click	outbound/UI click	link_url	engagement
user_engagement	engagement heartbeat	engagement_time_msec	engagement

The seven bolded stages plus `session_start` constitute the macro funnel (Section 10).

9.2 event_params Flattening Patterns

`event_params` is `ARRAY<STRUCT<key STRING, value STRUCT<string_value, int_value, float_value, double_value>>>`. The value lives in exactly one typed sub-field, so extraction

always targets the correct slot. The governed pattern is a correlated scalar subquery (avoids a fan-out join):

```
-- scalar extractors (used in staging)
(select ep.value.string_value from unnest(event_params) ep where ep.key = 'page_location') as
        page_location,
(select ep.value.int_value    from unnest(event_params) ep where ep.key = 'ga_session_id') as
        ga_session_id,
(select ep.value.double_value from unnest(event_params) ep where ep.key = 'engagement_time_msec') as
        engagement_time_msec
```

A reusable dbt macro standardizes this — the canonical `get_event_param` helper:

```
{% macro get_event_param(key, type='string') %}
  (select ep.value.{{ type }}_value from unnest(event_params) ep where ep.key = '{{ key }}')
{% endmacro %}
-- usage:  {{ get_event_param('ga_session_id','int') }} as ga_session_id
```

`stg_ga4__event_params` provides the fully-unnested alternative (one row per event x param), useful when you need to scan many keys at once:

```
-- stg_ga4__event_params: one row per (event, param key)
select
  to_hex(md5(user_pseudo_id||'-'||cast(event_timestamp as string)||'-'||event_name)) as event_key,
  user_pseudo_id, event_name, event_timestamp,
  ep.key as param_key,
  ep.value.string_value as string_value, ep.value.int_value as int_value,
  ep.value.float_value as float_value, ep.value.double_value as double_value
from {{ source('src_ga4','events') }}, unnest(event_params) ep
```

9.3 The items[] Array and the ecommerce Struct

`items` is `ARRAY<STRUCT< ... >>` carried on `view_item`, `add_to_cart`, `begin_checkout`, and `purchase`. Each element holds `item_id`, `item_name`, `item_brand`, `item_category` (+ `item_category2..5`), `item_variant`, `price_in_usd`, `price`, `quantity`, `item_revenue_in_usd`, `coupon`. To analyze items, `CROSS JOIN UNNEST(items)`:

```
select user_pseudo_id, i.item_id, i.item_name, i.item_category,
       i.quantity, i.item_revenue_in_usd, i.price_in_usd
from {{ source('src_ga4','events') }}, unnest(items) i
where event_name = 'purchase'
```

The scalar `ecommerce` struct (on `purchase`) carries order-level totals: `total_item_quantity`, `purchase_revenue_in_usd`, `purchase_revenue`, `refund_value_in_usd`, `shipping_value_in_usd`, `tax_value_in_usd`, `transaction_id`, `unique_items`. Order-level

revenue comes from `ecommerce.purchase_revenue_in_usd`; line-level revenue from the unnested `items.item_revenue_in_usd`. The two need not sum identically (order revenue excludes shipping/tax depending on config) — Section 11 resolves which is authoritative.

9.4 Example: extracting session id, page_location, source/medium

```
select
  user_pseudo_id,
  {{ get_event_param('ga_session_id','int') }}  as ga_session_id,
  {{ get_event_param('page_location') }}         as page_location,
  coalesce({{ get_event_param('source') }}, traffic_source.source) as session_source,
  coalesce({{ get_event_param('medium') }}, traffic_source.medium) as session_medium
from {{ source('src_ga4','events') }}
where event_name = 'page_view'
```

This is the only sanctioned path to source/medium: session-scoped `event_params` first, user first-touch `traffic_source` as fallback — honoring the gotcha.

---

10. Funnel Definitions

10.1 Canonical Macro Funnel

```
session_start → view_item → add_to_cart → begin_checkout → add_shipping_info → add_payment_info →
```

Helios reports **step-to-step** conversion (each stage / prior stage) AND **overall** `session_conversion_rate = purchasing_sessions / sessions`. The funnel is **session-scoped** and uses **max-downstream ("reached this step or beyond")** semantics: a session `reached_begin_checkout` if it fired `begin_checkout` OR any later funnel-stage event (`add_shipping_info`, `add_payment_info`, `purchase`) at any point in the session. This rolls each flag forward to every downstream stage, so the macro funnel is **MONOTONIC by construction**: `sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ reached_begin_checkout ≥ reached_add_shipping_info ≥ reached_add_payment_info ≥ reached_purchase`. Monotonicity prevents a downstream stage count from ever exceeding an upstream one (e.g. it guarantees `view_to_cart_rate ≤ 1` always holds) and avoids dropping sessions that, e.g., re-add a cart item without re-viewing the PDP. (An ordered variant is available for the Critic to test step-skipping hypotheses but is not the default.)

10.2 Micro-Funnels (Rates)

- `view_to_cart_rate` = `add_to_cart_sessions / view_item_sessions`
- `cart_to_checkout_rate` = `begin_checkout_sessions / add_to_cart_sessions`
- `checkout_to_purchase_rate` = `purchasing_sessions / begin_checkout_sessions`
- `cart_abandonment_rate` = `1 - (purchasing_sessions / add_to_cart_sessions)`
- `checkout_abandonment_rate` = `1 - (purchasing_sessions / begin_checkout_sessions)`

10.3 Session-Level Boolean Flags

```sql
-- int_ga4__funnel_steps : one row per session, max-downstream (monotonic) flags
-- each flag = "reached this step OR any later funnel stage"
select
  to_hex(md5(user_pseudo_id||'-'||cast(ga_session_id as string))) as session_key,
  user_pseudo_id,
  true
        as did_session_start,
  logical_or(event_name in
        ('view_item','add_to_cart','begin_checkout','add_shipping_info','add_payment_info','purchase'))
        as reached_view_item,
  logical_or(event_name in
        ('add_to_cart','begin_checkout','add_shipping_info','add_payment_info','purchase'))
        as reached_add_to_cart,
  logical_or(event_name in ('begin_checkout','add_shipping_info','add_payment_info','purchase'))
        as reached_begin_checkout,
  logical_or(event_name in ('add_shipping_info','add_payment_info','purchase'))
        as reached_add_shipping_info,
  logical_or(event_name in ('add_payment_info','purchase'))
        as reached_add_payment_info,
  logical_or(event_name = 'purchase')
        as reached_purchase
from (
  select user_pseudo_id, event_name,
         {{ get_event_param('ga_session_id','int') }} as ga_session_id
  from {{ source('src_ga4','events') }}
)
where ga_session_id is not null
group by user_pseudo_id, ga_session_id
```

10.4 Session vs User Scope; Ordered vs Unordered; Time Windows

Scope: the default funnel is session-scoped (denominator = `sessions`), which isolates within-visit friction. A user-scoped variant (denominator = `users`, "did this user ever purchase in window") is exposed for retention/LTV questions but never mixed with session rates in one finding — the Critic flags scope-mixing as a refutation.

Ordered vs unordered: default is unordered (occurrence). Ordered logic, when requested, requires the min-timestamp of each step to be monotonically increasing per session; it is strictly stricter and yields lower step rates.

Time window: funnels are computed inside a fixed analysis window (e.g. a week or the t0->t1 comparison windows the Decompose agent receives). A session is attributed to the day of its `session_start_micros`, so a session spanning midnight counts once, on its start day.

10.5 Worked Example — building fct_funnel

```
-- fct_funnel : session grain, flags + revenue, joined to session attributes
with steps as ( select * from {{ ref('int_ga4__funnel_steps') }} ),
sess  as ( select * from {{ ref('int_ga4__sessionized') }} ),
rev as (
  -- deduped session revenue: one purchase_revenue per distinct transaction_id
  select to_hex(md5(user_pseudo_id||'-'||cast(ga_session_id as string))) as session_key,
         sum(txn_rev) as session_revenue
  from (
    select user_pseudo_id,
           {{ get_event_param('ga_session_id','int') }} as ga_session_id,
           ecommerce.transaction_id as transaction_id,
           any_value(ecommerce.purchase_revenue_in_usd) as txn_rev
    from {{ source('src_ga4','events') }}
    where event_name='purchase' and ecommerce.transaction_id is not null
    group by 1,2,3   -- dedup duplicate purchase rows per transaction
  )
  group by session_key
)
select
  s.session_key, s.user_pseudo_id, date(timestamp_micros(s.session_start_micros)) as date_key,
  c.channel_key, s.device_category, s.country, (s.ga_session_number = 1) as is_new_user,
  st.did_session_start, st.reached_view_item, st.reached_add_to_cart, st.reached_begin_checkout,
  st.reached_add_shipping_info, st.reached_add_payment_info, st.reached_purchase,
  coalesce(r.session_revenue, 0.0) as session_revenue
from sess s
join steps st using (session_key)
left join rev r using (session_key)
left join {{ ref('dim_channels') }} c on c.channel_group = {{
        derive_channel_group('s.source','s.medium') }}
```

`session_conversion_rate` is then `countif(reached_purchase) / count(*)` over `fct_funnel`, always recomputed from additive counts so it re-aggregates correctly across any dimension the Decompose agent slices.

---

11. Revenue Definitions

11.1 Precise Metric Definitions

- **gross_revenue** = `SUM(purchase_revenue_in_usd)` over `purchase` events, **deduped by transaction_id** (one revenue figure per distinct `transaction_id`).

- **net_revenue** = `gross_revenue − SUM(refund_value_in_usd)`.
- **revenue** (canonical) = `gross_revenue` (the headline figure unless a finding explicitly concerns refunds).
- **item_revenue** = `SUM(items.item_revenue_in_usd)` over unnested purchase items.
- **transactions** = `COUNT(DISTINCT transaction_id)`.
- **aov** = `revenue / transactions`.
- **items_per_transaction** = `SUM(total_item_quantity) / transactions`.
- **revenue_per_session** (RPS) = `revenue / sessions`.
- **revenue_per_user** (ARPU) = `revenue / users`.

11.2 Currency, Shipping, and Tax

All money uses the `_in_usd` fields (`purchase_revenue_in_usd`, `item_revenue_in_usd`, `refund_value_in_usd`, `shipping_value_in_usd`, `tax_value_in_usd`, `price_in_usd`). The non-USD twins (`purchase_revenue`, `price`) are ignored to avoid mixed-currency aggregation. `purchase_revenue_in_usd` is product revenue **excluding shipping and tax**; shipping and tax are stored separately on `fct_orders` and never folded into `revenue` (so `aov` reflects merchandise value). The `_in_usd` fields are GA4's normalized columns, so no FX conversion is performed by Helios.

11.3 Dedup Gotchas

GA4 exports can emit **duplicate purchase rows** for one `transaction_id` (retries, multi-stream). Summing `purchase_revenue_in_usd` raw double-counts. The fix is to collapse to one row per `transaction_id` first (`any_value` of the revenue), then sum. **NULL transaction_id** rows are purchases that GA4 failed to tag (test orders, mis-instrumented) — they are excluded from `transactions` and `gross_revenue` but logged by the Critic as a data-quality caveat, since a spike in NULL-id purchases can masquerade as a revenue drop.

11.4 Refunds

Refunds appear as a non-null `refund_value_in_usd` on the purchase event (this dataset rarely has a separate `refund` event). `net_revenue` subtracts it; in practice refunds are near-zero in the sample window, so `gross_revenue ~= net_revenue`, but the distinction is preserved for the revenue-at-risk dollar quantification every finding carries.

11.5 SQL — fct_orders revenue

```
-- fct_orders : one deduped row per transaction_id
with purch as (
  select
    ecommerce.transaction_id as order_key,
```

```
    to_hex(md5(user_pseudo_id||'-'||cast(
       (select ep.value.int_value from unnest(event_params) ep where ep.key='ga_session_id') as
        string))) as session_key,
    user_pseudo_id,
    timestamp_micros(event_timestamp) as order_ts,
    any_value(ecommerce.purchase_revenue_in_usd) as gross_revenue,
    any_value(ecommerce.refund_value_in_usd)     as refund_value_in_usd,
    any_value(ecommerce.shipping_value_in_usd)   as shipping_value_in_usd,
    any_value(ecommerce.tax_value_in_usd)        as tax_value_in_usd,
    any_value(ecommerce.total_item_quantity)     as total_item_quantity,
    any_value(ecommerce.unique_items)            as unique_items
  from {{ source('src_ga4','events') }}
  where event_name = 'purchase' and ecommerce.transaction_id is not null
  group by order_key, session_key, user_pseudo_id, order_ts   -- collapse duplicate purchase rows
)
select
  order_key, session_key, user_pseudo_id, date(order_ts) as date_key, order_ts,
  gross_revenue,
  coalesce(refund_value_in_usd, 0.0) as refund_value_in_usd,
  gross_revenue - coalesce(refund_value_in_usd, 0.0) as net_revenue,
  coalesce(shipping_value_in_usd,0.0) as shipping_value_in_usd,
  coalesce(tax_value_in_usd,0.0) as tax_value_in_usd,
  total_item_quantity, unique_items
from purch
```

11.6 SQL — headline revenue metrics

```
select
  count(distinct order_key)                                          as transactions,
  sum(gross_revenue)                                                 as gross_revenue,
  sum(net_revenue)                                                   as net_revenue,
  safe_divide(sum(gross_revenue), count(distinct order_key))         as aov,
  safe_divide(sum(total_item_quantity), count(distinct order_key))   as items_per_transaction
from {{ ref('fct_orders') }}
-- revenue_per_session / revenue_per_user join fct_orders to fct_sessions / dim_users:
-- revenue_per_session = sum(gross_revenue) / count(distinct session_key from fct_sessions)
-- revenue_per_user    = sum(gross_revenue) / count(distinct user_pseudo_id from dim_users)
```

These definitions are the single source consumed by `semantic-mcp.get_metric`; any deviation (a synonym, an un-deduped sum, mixing `purchase_revenue` with `_in_usd`) is rejected at the semantic layer and surfaced by the Critic, upholding the 100%-governed-SQL target.